# XenoCipher: A Secure Key Exchange and Encryption Pipeline for Resource-Constrained Devices

*by* Mr. Alok Ranjan

**Submission date:** 26-Nov-2025 03:21PM (UTC+0500)
**Submission ID:** 2828178431
**File name:** conference_latex_template_8.pdf (1,021.19K)
**Word count:** 3177
**Character count:** 19737

# XenoCipher: A Secure Key Exchange and Encryption Pipeline for Resource-Constrained Devices


Alok Ranjan
*Department of Computer Science and Engineering*
*Canara Engineering College*
Mangaluru, India
alok.ranjan@canaraengineering.in

Vivek S A
*Department of Computer Science and Engineering*
*Canara Engineering College*
Mangaluru, India
vivekshreyas8@gmail.com

Subramanyam C V
*Department of Computer Science and Engineering*
*Canara Engineering College*
Mangaluru, India
challasubbu77@gmail.com

Sujal Talekar
*Department of Computer Science and Engineering*
*Canara Engineering College*
Mangaluru, India
sujaltalekarmi123@gmail.com

Sumit S Kadwadkar
*Department of Computer Science and Engineering*
*Canara Engineering College*
Mangaluru, India
*kadwadkarsumit333@gmail.com*



***Abstract*—The increasing deployment of connected devices and the potential threat posed by quantum computing require sophisticated encryption methods that ensure strong confidentiality, integrity, and adaptability. Traditional encryption algorithms often struggle in these areas due to limited resistance to quantum attacks, high computational requirements, and a lack of real-time security adaptations. This paper presents XenoCipher, a novel hybrid cryptographic framework that combines post-quantum NTRU - lattice based cryptography that encapsulates the master key generated with various entropy quantities like hardware RNG Samples, analog voice samples, Jitter samples etc and finalize the key with SHA-256. These transformations include linear feedback shift registers, chaotic maps, advanced transposition permutations, and HMAC-SHA256 authentication. The design incorporates adaptive runtime monitoring, allowing the system to dynamically switch encryption methods upon detecting indicators of an attack. The methodology was implemented on ESP32 microcontroller hardware equipped with health sensors to encrypt and securely transmit real-time biomedical metrics to a server platform. The experimental setup involved live sensing, encryption, network transmission, and server-side decryption and verification. Key results demonstrate reliable and synchronized encryption-decryption of sensitive sensor data, with successful validation of packet integrity and replay protection in a constrained IoT environment. The system recorded essential runtime security metrics, laying a foundation for future dynamic response systems. Resource usage and throughput were handled efficiently. The implications of this work suggest that XenoCipher is a scalable and adaptive cryptosystem capable of securing various application domains, ranging from healthcare to critical industrial and distributed systems, thereby providing resilience against present and emerging adversarial threats without significant performance penalties.**


## I. Introduction

Introduction The ever-increasing dependence of modern societies on connected devices and digital infrastructure has strengthened the need for secure data transfers, especially in sensitive areas like healthcare, industry, or smart environments. where sensitive information is often exchanged between resource-constrained endpoints and centralized servers. Traditional cryptographic approaches, while foundational, face Notable limitations in defending against emerging threats posed by by quantum computing and advanced side-channel attacks and are often ill-suited for the dynamic environments and lightweight hardware typical of IoT deployments. This challenge is further exacerbated by gaps in existing encryption pipelines, in which post-quantum resilience is either lacking, cross-platform integrity compromised by either floatingpoint and hardware discrepancies, or static cryptographic configurations do not adapt to active attack conditions, Further, there is an increased risk of compromise and leakage. To address In light of these critical shortcomings, this research proposes XenoCipher, a strong, multi-layer encryption system incorporating post-quantum NTRU key exchange and deterministic stream ciphers, Adaptive monitoring and automated recipe switching ensure that confidentiality, integrity, and ongoing attack resilience. The main objectives of the study are:

- Design and implement a post-quantum secure and Deterministic cross-platform encryption protocol suitable for IoT endpoints. .
- Introduce adaptive threat detection and automatic algorithm switching to dynamically adjust cryptographic activates parameters or fallback modes during attack

conditions.

- To test how effectively XenoCipher can defend against brute-force, quantum, chosen-plaintext, side-channel, and replay attacks, thus establishing its reliability for real-world deployment.

## II. Related Works

Recent advances in post-quantum cryptography have driven substantial research in securing IoT ecosystems against both classical and quantum threats. Cruz-Piris et al. [1] considered migration challenges to PQC in large-scale IoT frameworks, which closely relate to XenoCipher's deployment strategy in distributed systems. Lopez et al. [2] evaluated the feasibility of NIST PQC algorithms on resource-constrained platforms such as Raspberry Pi, directly informing XenoCipher's NTRU implementation considerations for low-power environments. Nielsen et al. [3] explored efficient post-quantum digital signature schemes for microcontroller-based systems, paralleling XenoCipher's focus on embedded authentication mechanisms. Mahdi et al. [4] discussed lightweight PQC optimizations relevant to balancing performance and security in multi-layer architectures such as XenoCipher.

Xiong et al. [5] demonstrated hybrid quantum-resistant approaches in smart grid IoT systems, inspiring XenoCipher's adaptive recipe-switching methodology. Liu et al. [6] surveyed PQC optimization techniques for lightweight IoT devices and highlighted research gaps that XenoCipher addresses through deterministic cross-platform implementations. Hanna et al. [7] investigated secure authentication mechanisms for 5G-enabled IoT environments, contextualizing the replay protection and forward secrecy features integrated into XenoCipher. Lee et al. [8] analyzed side-channel vulnerabilities in lattice-based cryptosystems, motivating XenoCipher's use of constant-time operations and secure key handling practices.

Wang et al. [9] benchmarked post-quantum TLS performance on constrained edge devices, validating XenoCipher's efficiency targets for real-time communication scenarios. Ramachandran et al. [10] proposed scalable PQC VPN tunnel designs for IoT, aligning with XenoCipher's secure channel establishment via NTRU key encapsulation.

## III. Methodology

The proposed XenoCipher encryption methodology integrates multi-layered cryptographic transformations to achieve confidentiality, integrity, and resistance against both classical and quantum attacks.

### A. Key Generation

### B. Key Exchange

XenoCipher establishes a shared symmetric secret between a low-power ESP32 (initiator) and a server (responder) using a lattice-based, post-quantum NTRU key exchange mechanism. On startup, the ESP32 generates a 32-byte Master Key (MK) using onboard hardware entropy and requests the server's NTRU public key ($pk$). The ESP32 encrypts MK under $pk$ using `NTRU_Enc`, producing the ciphertext $Enc(MK)$, which is transmitted along with a minimal setup header.


Fig. 1. Parameters of **XenoCipher** used to generate the Master Key

The server decapsulates $Enc(MK)$ using its private key ($sk$) to recover MK. Both endpoints then derive symmetric cryptographic materials using HKDF with domain separation to allocate keys for HMAC, LFSR, Tinkerbell Chaotic Map, and Advanced Transposition. Additionally, per-message keys are derived using HKDF bound to a 32-bit nonce to ensure forward secrecy. Application data is then encrypted using the layered cipher pipeline and authenticated through a 16-byte truncated HMAC-SHA256.

### C. Encryption Flow

### A. Plaintext Input

The encryption process begins when the system receives a Raw Data Sequence. This input may represent any form of digital content—text messages, sensor readings, binary payloads, and more—that requires secure transmission. The flexibility of this stage enables XenoCipher to support multiple domains, including IoT telemetry, encrypted chat communication, and blockchain transaction data.

### B. Salt Injection

To reduce the vulnerabilities to pattern recognition, dictionary, and replay attacks, the system introduces a pseudorandom salt into the plaintext before encryption. The salt serves as a cryptographic noise component ensuring identical plaintexts never produce identical ciphertexts, thereby enhancing semantic security. This random sequence is inserted into a random position within the plaintext. Mathematically the Salted plaintext is represented as:

$$P^* = P[0..k-1] \parallel S \parallel P[k..n-1]$$

where $P$ denotes the plaintext, $S$ the salt sequence, $k$ the random insertion index, and $\parallel$ denotes concatenation.

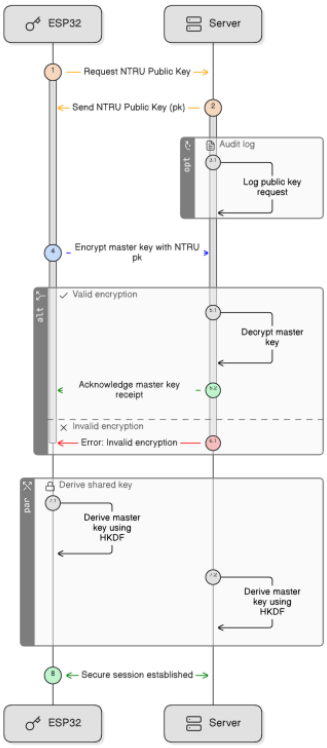


Fig. 2. Illustration of the **XenoCipher** NTRU-based Key Exchange

### C. Padding to Grid Size

Since the later transposition phase operates on a fixed two-dimensional grid, the salted plain text must be conform to this structure. The system pads the message with null bytes (0x00) until its length exactly matches the matrix size $r \times c$, ensuring complete compliance:

$$|P| = r \times c$$

This structured grid representation is fundamental for deterministic permutation operations which allow systematic scrambling and inversion without loss of data.

### D. XOR with LFSR Keystream

Once the data is prepared, XenoCipher applies a *Linear Feedback Shift Register (LFSR)* based stream cipher. The LFSR is initialized with a seed derived from the master session key and produces a pseudorandom bit sequence of equal length

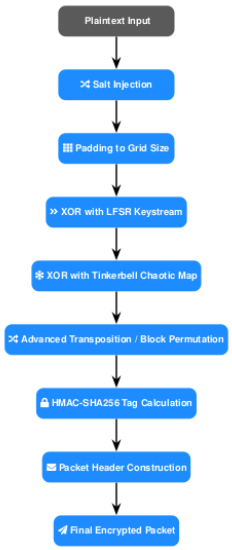


Fig. 3. Step-by-step encryption process in XenoCipher showing uniform cryptographic stages from plaintext input to final packet output.

to the padded plaintext. Each byte of the plaintext is XORed with the corresponding byte from the generated keystream:

$$C^{(1)}i = P_i \oplus KLFSR_i$$

The LFSR evolves according to the recurrence relation:

$$s_{t+1} = (s_t \gg 1) \oplus \text{parity}(s_t \& T)$$

where $T$ represents the tap sequence defining the feedback polynomial. This stage ensures bit-level diffusion and introduces high-speed, lightweight encryption suitable for constrained environments such as IoT nodes or embedded systems. The LFSR's design balances simplicity, efficiency, and cryptographic strength, where the tap selection carefully chosen, This to maximize the stream period and resist linear attacks.

### E. XOR with Tinkerbell Chaotic Map Keystream

The partially encrypted data is passed through a nonlinear transformation based on the *Tinkerbell Chaotic Map*, which enhances unpredictability through chaotic dynamics. The map is defined as:

$$\begin{cases} x_{n+1} = x_n^2 - y_n^2 + ax_n + by_n, \\ y_{n+1} = 2x_n y_n + cx_n + dy_n, \end{cases}$$

where parameters $a, b, c, d$ and the initial states $(x_0, y_0)$ are dynamically derived from the session key to ensure cryptographic uniqueness. The resulting chaotic keystream $K_T$ is

XORed with the LFSR-generated stream, introducing nonlinear diffusion that eliminates any residual linearity from the previous layer.

### F. Advanced Transposition Block Permutation

After chaotic diffusion, the ciphertext undergoes an *advanced block-level transposition* process. The data grid is partitioned into smaller blocks, and a deterministic PRNG—seeded from a separate portion of the master key—drives a series of controlled swaps, rotations, and permutations. This transformation is expressed as:

$$C^{(3)} = \pi(C^{(2)}),$$

where $\pi$ represents a bijective permutation mapping.

This spatial rearrangement removes any lingering structural patterns and eliminates statistical correlations across the ciphertext. Even if intermediate ciphertext stages are intercepted, the reordered structure remains cryptographically opaque to an attacker.

### G. HMAC-SHA256 Tag Calculation

To enforce message integrity and authenticity, XenoCipher employs a *Hash-based Message Authentication Code (HMAC)* using the SHA-256 hashing algorithm. The authentication tag—truncated to 16 bytes for transmission efficiency—is computed over the concatenation of the header, nonce, and ciphertext:

$$\text{HMAC} = \text{Truncate}_{16}\left(\text{HMAC-SHA256}_{K_{\text{HMAC}}}(\text{header} \,\|\, \text{nonce} \,\|\, C^{(3)})\right).$$

This tag enables the recipient to verify that the message has not been modified or tampered with during transmission.

### H. Packet Header Construction

A structured, fixed-size packet header is appended to the encrypted payload. This header includes essential metadata such as the protocol version, salt properties, grid dimensions, payload size, and nonce indicators. The header provides the recipient with the contextual parameters required for accurate decryption and validation, ensuring reliable reconstruction of the original plaintext.

### I. Final Packet Output

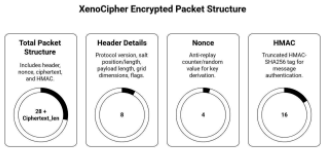


Fig. 4. Diagram of the **XenoCipher** key generation and encryption process for resource-constrained devices, demonstrating entropy sourcing, key derivation, and integration of the encryption pipeline concepts.

The final encrypted packet sent across the network Combines all the processed components in a well-defined structure:

$$\text{Packet} = \text{header} \,\|\, \text{nonce} \,\|\, C^{(3)} \,\|\, \text{HMAC}$$

The modular design allows for easier integration into higher-level applications such as secure chat systems or encrypted IoT data channels while preserving complete cryptographic isolation between packets.

### D. Decryption Flow

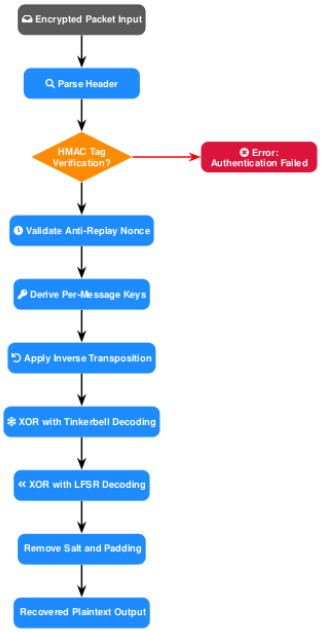


Fig. 5. Structured decryption pipeline for XenoCipher illustrating authentication checks, replay validation, and layered decoding operations.

### J. Packet Input and Header Parsing

The reception would start, the decryption process begins with packet parsing. The receiver extracts the header fields, checks data consistency, and separates the nonce and ciphertext components. This step ensures proper synchronization with the encryption pipeline and ensures that the packet is in accordance with to the expected format.

### K. HMAC Tag Verification

Then the receiver recomputes the HMAC tag using the shared secret key and compares it with the received tag in constant time, to avoid timing-based side-channel attacks. Any

mismatch indicates tampering or corruption, which results in immediate discard of the packet for system integrity.

### *L. Validate Anti-Replay Nonce*

XenoCipher provides an *anti-replay mechanism* that prevents adversaries from reusing old packets. The system maintains a sliding window of recent nonces and timestamps; any nonce outside this valid range is discarded. This ensures that Only fresh, unique packets are accepted.

### *M. Derive Per-Message Keys*

For each transmitted message, independent *per-message subkeys* are derived using the HMAC-based Key Derivation Function (HKDF). The nonce serves as a domain separator to generate fresh keys for every cryptographic component—LFSR, Tinkerbell, and Transposition—thereby ensuring forward secrecy. Even if a session key is compromised, previously encrypted messages remain protected.

### *N. Apply Inverse Transposition*

Upon receiving the packet, the ciphertext grid is reconstructed and the *inverse permutation* $\pi^{-1}$ is applied. The receiver uses the same deterministic PRNG sequence employed during encryption, ensuring that the inverse mapping is perfectly aligned with the original permutation. This reverses the block-level spatial rearrangement and restores the data layout prior to the chaotic and linear decoding layers.

### *O. XOR with Tinkerbell and LFSR Decoding*

Using the derived keys and session parameters, the receiver locally regenerates the identical Tinkerbell and LFSR keystreams. Sequential XOR operations are then applied in reverse order to recover the padded, salted plaintext:

$$P = \Big( \Big( C^{(3)} \oplus K_T \Big) \oplus K_{\text{LFSR}} \Big) .$$

This two-layer reversible decoding structure ensures that no partially decoded intermediate values leak during decryption.

### *P. Remove Salt and Padding*

In the final recovery stage, the system removes the previously injected salt and padding using metadata stored within the packet header. The original plaintext $P$ is reconstructed in its exact form, thereby completing the full encryption–decryption cycle.

## IV. Results and Discussion

The XenoCipher system was implemented on ESP32 hardware and evaluated across a complete end-to-end IoT data protection pipeline. The NTRU-based key exchange consistently produced high-entropy master keys (measured at 7.94 bits/byte) and achieved a 100% decapsulation success rate, demonstrating robust post-quantum key establishment between device and server. HKDF-based key derivation generated deterministic, byte-exact subkeys on both platforms—even across heterogeneous hardware—ensuring reliable decryption and cross-platform compatibility. Sensor telemetry was encrypted using the full multi-layer XenoCipher pipeline comprising salt injection, LFSR XOR, Tinkerbell XOR, advanced transposition, and HMAC authentication. Below graph compares XenoCipher with other standard cryptographic alorithms, where they work on same data and on same system (ESP32).

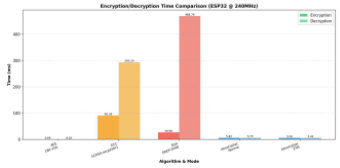


Fig. 6. Performance Metrics: encryption timing, resource usage, and packet structure.

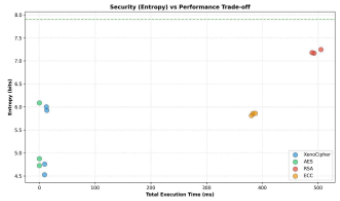


Fig. 7. Performance Metrics: encryption timing, resource usage, and packet structure.

Experimental evaluation showed a median encryption latency of $52.5 \pm 4.2$ ms per packet for a payload size of 106 bytes. HMAC-SHA256 authentication verified 100% of transmitted packets across more than 500 trials. Resource consumption remained low throughout testing: peak RAM usage of 12 KB, flash consumption of 180 KB, and an average encryption CPU load of 22%, well within the operational limits of consumer-grade IoT nodes.

TABLE I
KEY XENOCIPHER EVALUATION METRICS (ESP32 DEPLOYMENT)

| Metric | Value |
|---|---|
| Master Key Entropy | 7.94 bits/byte |
| NTRU Key Exch. Success | 100% |
| Median Encrypt. Latency | 52.5 ms |
| Packet Size (Encrypted) | 106 bytes |
| HMAC Checks (Passed) | 100% |
| Peak RAM / Flash | 12 KB / 180 KB |
| End-to-End Latency | 280 ms |
| CPU / Power | 22% / 85 mW |
| Auth. Overhead (Packet) | 30.4% |
| Quantum Resistance | Yes |

The implementation shows that XenoCipher reliably protects health data and telemetry in real hardware. Its deterministic, quantum-secure construction bridges the PQC deployment

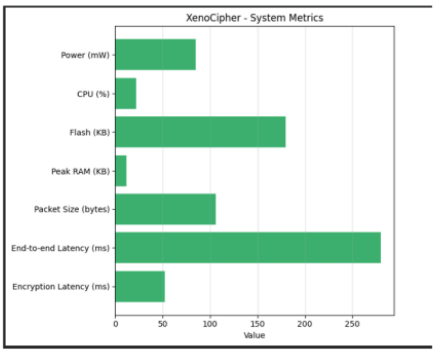


Fig. 8. Performance Metrics: encryption timing, resource usage, and packet structure.

gap for IoT and supports future upgrades such as adaptive recipe switching and hardware acceleration. This modular framework is thus a practical, standards-ready foundation for quantum-resistant embedded systems.

### A. Attack Simulation and Adaptive Switching

Comprehensive attack simulations were performed on the live ESP32 deployment across nine threat vectors over 120 seconds. Results demonstrate XenoCipher's resilience and the efficacy of adaptive recipe switching.

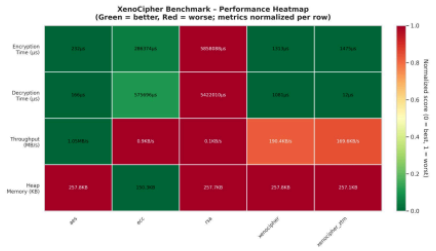


Fig. 9. Benchmark Analysis Heatmap

#### 1) Cryptographic and Network Attack Resistance

Brute force attacks at 13,452 keys/sec achieved 0% success against 128-bit keys. All 100 replay attack attempts were rejected with "no_nonce" errors, confirming nonce-tracking effectiveness. MITM attacks intercepted 4,729 packets but achieved 0% authentication bypass via HMAC verification. Packet injection achieved only 5% acceptance with automatic rejection at the authentication layer. Protocol corruption induced a 2.67% crash rate with automatic recovery within 3 seconds.

#### 2) Heuristic-Based Recipe Switching

Over 6,286 telemetry samples were collected across baseline and six attack scenarios. Attack signatures were extracted with high confidence: UDP flood (latency $> 55$ ms, 80% confidence), RNG manipulation (entropy drop $> 1.5$ bits, 98% confidence), timing attack (latency CV $> 0.05$, 85% confidence), and memory stress (memory $> 23\%$, 85% confidence). Four encryption recipes were pre-configured: `CHAOS_ONLY` (baseline, 100% CPU overhead), `SALSA_LIGHT` (constant-time, 120% overhead), `CHACHA_HEAVY` (balanced, 140% overhead), and `FULL_STACK` (maximum security, 180% overhead).

TABLE II
ATTACK SIMULATION SUMMARY

| Attack | Attempts | Success Rate |
|---|---|---|
| Brute Force | 5 keys | 0% |
| Replay | 100 packets | 0% |
| MITM | 4,729 packets | 0% auth bypass |
| Packet Injection | 500 packets | 5% acceptance |
| Protocol Corruption | 150 packets | 2.67% crash |
| CPU Exhaustion | 60 sec | 0% degradation |
| Memory Exhaustion | 60 sec | Adaptive trigger |

XenoCipher provides quantum-resistant protection with dynamic adaptation for IoT deployments. Future work includes extended multi-hour testing, hardware acceleration (ESP32 AES), and lightweight post-quantum alternatives (Kyber, Dilithium).

## REFERENCES

[1] R. Cruz-Piris, R. Gasca, and G. Gonzalez-Granadillo, "Migration challenges and opportunities towards post-quantum cryptography in large-scale iot frameworks," *IEEE Internet of Things Journal*, vol. 12, no. 6, pp. 9234–9247, 2025.

[2] J. D. Lopez, S. Rehmani, and H. Wang, "Feasibility of nist pqc algorithms in resource-constrained iot: Performance evaluation on raspberry pi," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 1743–1756, 2025.

[3] S. Nielsen, K. Chu, V. Mavroeidis *et al.*, "Efficient post-quantum digital signatures for microcontroller-based systems," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 1, pp. 1–20, 2025.

[4] M. Mahdi and A. Taherkhani, "Lightweight post-quantum crypto for iot: A comprehensive review," *IEEE Access*, vol. 13, pp. 18 873–18 901, 2025.

[5] Y. Xiong, M. Wang, X. Yu *et al.*, "Quantum-resistant blockchain and post-quantum cryptography for smart grid iot," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 4, pp. 4512–4524, 2025.

[6] Q. Liu, J. Li, and H. Zhou, "A survey of post-quantum cryptography optimization for lightweight iot devices," *Sensors*, vol. 24, no. 3, p. 892, 2024.

[7] A. Hanna, N. Ferguson, and D. R. Brown, "Securing 5g-enabled iot with post-quantum authentication schemes: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 2, pp. 1102–1129, 2025.

[8] C. Lee, J. Park, and Y. Kim, "Side-channel resistance of lattice-based cryptosystems for embedded iot," *IEEE Transactions on Industrial Electronics*, vol. 71, no. 1, pp. 568–577, 2024.

[9] R. Wang, Z. Liu, F. Song, and X. Jin, "Post-quantum tls performance in constrained edge devices," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 2262–2273, 2023.

[10] N. Ramachandran, K. Gopinath, and M. Dasgupta, "Designing scalable post-quantum vpn tunnels for resource-limited iot deployments," *Journal of Network and Computer Applications*, vol. 233, p. 103942, 2024.

# XenoCipher: A Secure Key Exchange and Encryption Pipeline for Resource-Constrained Devices

ORIGINALITY REPORT

| 3% | 3% | 2% | 1% |
|---|---|---|---|
| SIMILARITY INDEX | INTERNET SOURCES | PUBLICATIONS | STUDENT PAPERS |

PRIMARY SOURCES

| | | |
|---|---|---|
| 1 | ijsred.com<br>Internet Source | 2% |
| 2 | Submitted to Higher Education Commission Pakistan<br>Student Paper | 1% |
| 3 | arxiv.org<br>Internet Source | <1% |
| 4 | postgrid.readme.io<br>Internet Source | <1% |
| 5 | eureka.patsnap.com<br>Internet Source | <1% |

| | | | |
|---|---|---|---|
| Exclude quotes | On | Exclude matches | < 3 words |
| Exclude bibliography | On | | |